

继承-派生类

一.继承概述

1.软件复用

在开发一个新的软件时，把现有软件的一些功能拿过来用称为软件复用

- 可复用的软件部件是子程序（函数），或类
- 软件复用时需要根据要求进行适当的修改

2.继承

定义一个新的类，先把已有的一个或多个类的功能全部包含进来，然后再在新的类中给出新功能的定义或对已有类的某些功能进行重新定义。

- 基类（父类）、派生类（子类）
- 派生类含有基类的所有成员，并可以定义新的成员和对基类的一些成员进行重新定义
- 单继承：一个类只有一个直接基类
- 多继承：一个类有多个直接基类

继承机制除了支持软件复用，还可以对处理的对象按层次进行分类、对概念进行组合、支持软件的增量开发。

3.单继承

1.格式：

```
class<派生类名> : [ <继承方式> ] <基类名>
```

```
{<成员说明表>
```

```
};
```

```
1  class A//基类
2  {   int x,y;
3      public:
4          void f();
5          void g();
6  };
7  class B:public A//派生类
8  {   int z;//新成员
9      public:
10         void h();//新成员
11  };
```

- 派生类除了拥有新定义的成员，还拥有基类的所有成员（基类的构造函数、析构函数和赋值操作符重载函数除外）。
- 派生类的对象包含了基类的一个子对象，该对象的内存空间位于派生类对内存空间的前部。
- 定义派生类时一定要见到基类的定义。
- 友元：

o 如果在派生类由没有显式指出，则基类的友元不是派生类的友元

○ 如果基类是另一个类的友元，而该基类没有显式指出，则该派生类不是该类的友元。

- 派生类不能直接访问基类的私有成员

```
1 class A
2 {   int x,y;
3   public:
4     void f();
5     void g() { ... x , y ... }
6 };
7
8 class B: public A
9 {   int z;
10  public:
11    void h()
12    { ... x, y ... //Error, x、y为基类的私有成员
13      f(); //OK
14      g(); //OK, 通过函数g访问基类的私有成员x和y
15    }
16 };
17
```

继承与封装的矛盾!

这样的话，一个类的成员有两种被外界使用的场合：

- 通过类的对象（实例）使用
- 在派生类中使用

```
1 class A
2 { .....
3     m
4 };
5 class B: public A
6 { .....
7     f() { ... m ... } //通过派生类使用A的成员m
8 };
9 void g()
10 { A a;
11     ... a.m ... //通过A的对象（实例）使用A的成员m
12 }
13
```

2.protected访问控制

- 用protected说明的成员不能通过对象使用，但可以在派生类中使用
- c++类向外界提供两种接口：
 - public:供类的实例用户使用（通过对象）
 - public+protected:供派生类使用

3.派生类成员标识符的作用域

对基类而言，派生类成员标识符的作用域是嵌套在基类作用域中的。

```

1  class A
2  { .....
3  public:
4      void f() { g(); } //Error!
5      .....
6  }
7
8  class B:public A
9  { .....
10 public:
11     void g() { f(); } //OK
12 }
13

```

- 如果派生类中定义了与基类同名的成员，则基类的成员名在派生类的作用域内不直接可见（被隐藏），访问基类同名的成员时要用基类名受限

```

1  class A//基类
2  {   int x,y;
3  public:
4      void f();
5      void g();
6  };
7
8  class B: public A
9  {   int z;
10 public:
11     void f(); //不是重载A的f!
12     void h()
13     {   f(); //B类中的f
14         A::f(); //A类中的f
15     }
16 };
17

```

注意：B类中的f与A类中的f不属于函数名重载，因为它们属于不同的作用域

- 即使派生类中定义了与基类同名但参数不同的成员函数，基类的同名函数在派生类的作用域中也是不直接可见的，仍然需要用基类名受限方式来使用之：

```

1  class A //基类
2  {   int x,y;
3  public:
4      void f();
5      void g();
6  };
7
8  class B: public A
9  {   int z;
10 public:
11     void f(int);
12     void h()
13     {   f(1); //OK
14         f(); //Error

```

```

15     A::f(); //OK
16 }
17 };
18

```

- 也可以在派生类中使用using声明把基类中的某个的函数名对派生类开放：

```

1  class A //基类
2  {   int x,y;
3  public:
4      void f();
5      void g();
6  };
7
8  class B: public A
9  {   int z;
10 public:
11     using A::f;
12     void f(int);
13     void h()
14     {   f(1); //OK
15         f(); //OK, 等价于A::f();
16     }
17 };
18

```

4.继承方式

问题：基类成员在派生类中对外的访问控制，派生类的用户能访问基类的那些成员？

1.继承方式可以是：public,private和protected

默认的继承方式为：private

基类成员 派生类 继承方式	public	private	protected
public	public	不可访问	protected
private	private	不可访问	private
protected	protected	不可访问	protected

```

1  class A
2  {public:
3      void f();
4  protected:
5      void g();
6  private:
7      void h();
8  };
9  class B: protected A
10 { //f为protected
11     //g为protected

```

```

12 //h为不可访问
13 public:
14     void q()
15     { f(); //?
16       g(); //?
17       h(); //?
18     }
19 };
20
21 class C: public B
22 {public:
23     void r()
24     { f(); //OK
25       g(); //OK
26       h(); //Error
27       q(); //?
28     }
29 };
30
31 void func()
32 { B b;
33   b.f(); //Error
34   b.g(); //Error
35   b.h(); //Error
36   b.q(); //?
37 }
38

```

- 继承方式的调整

```

1 class A
2 { public:
3     void f1();
4     void f2();
5     void f3();
6     protected:
7     void g1();
8     void g2();
9     void g3();
10 };
11
12 class B: private A
13 { public:
14     A::f1(); //把f1调整为public
15     A::g1(); //把g1调整为public
16     //是否允许弱化基类的访问控制要视具体的实现而定
17     protected:
18     A::f2(); //把f2调整为protected
19     A::g2(); //把g2调整为protected
20     .....
21 };
22

```

2.public继承与子类型

- public继承有特殊的作用，它可以实现类之间的子类型关系：
 - 对用类型T表达的所有程序P，当用类型S去替换程序P中的所有的类型T时，程序P的功能不变，则称类型S是类型T的子类型
- 在C++中，把类看作类型，把以public方式继承的派生类看作是基类的子类型：
 - 对基类对象能实施的操作也能作用于派生类对象
 - 在需要基类对象的地方可以用派生类对象去替代

```

1  例如，对于下面的两个类A和B
2  class A //基类
3  {      int x,y;
4  public:
5      void f() { x++; y++; }
6      .....
7  };
8
9  class B: public A //派生类
10 {      int z;
11 public:
12     void g() { z++; }
13     .....
14 };
15
16 下面的操作是合法的：
17  A a;
18  B b;
19  b.f(); //OK，基类的操作可以实施到派生类对象
20  a = b; //OK，派生类对象可以赋值给基类对象，
21      //属于派生类但不属于基类的数据成员将被忽略
22  A *p=&b; //OK，基类的指针可以指向派生类对象
23  A &a2=b; //OK，基类的引用可以引用派生类对象
24  .....
25  void func1(A *p);
26  void func2(A &x);
27  void func3(A x);
28  func1(&b); func2(b); func3(b); //OK
29
30 下面的操作是不合法的：
31  A a;
32  B b;
33  a.g(); //Error，基类对象a没有g这个成员函数
34  b = a; //Error，它将导致b有不一致的成员数据
35      //(a中没有属于派生类的数据)
36  B *q=&a; //Error，“q->g();”将修改不属于a的数据！
37  B &b2=a; //Error，“b2.g();”将修改不属于a的数据！
38  .....
39  void func1(B *p);
40  void func2(B &x);
41  void func3(B x);
42  func1(&a); func2(a); func3(a); //Error
43

```

二.派生类

1.派生类对象的初始化和消亡处理

- 派生类对象的初始化由基类和派生类共同完成：
 - 从基类继承的数据成员由基类的构造函数初始化；
 - 派生类新的数据成员由派生类的构造函数初始化。
- 当创建派生类的对象时：
 - 先执行基类的构造函数，再执行派生类构造函数。
 - 默认情况下，调用基类的默认构造函数，如果要调用基类的非默认构造函数，则必须在派生类构造函数的成员初始化表中显式指出。
- 当派生类对象消亡时：
 - 先调用本身类的析构函数，执行完后会自动调用基类的析构函数。

```
1  class A
2  {   int x;
3  public:
4      A() { x = 0; }
5      A(int i) { x = i; }
6  };
7
8  class B: public A
9  {   int y;
10 public:
11     B() { y = 0; }
12     B(int i) { y = i; }
13     B(int i, int j):A(i) { y = j; }
14 };
15 .....
16
17 B b1;      //执行A::A()和B::B(), b1.x等于0, b1.y等于0。
18 B b2(1);   //执行A::A()和B::B(int), b2.x等于0, b2.y等于1。
19 B b3(1,2); //执行A::A(int)和B::B(int,int), b3.x等于1,
20           //b3.y等于2。
21
```

对一个未提供任何构造函数的类，如果它有基类，编译程序有时会隐式地为之提供一个默认构造函数，其作用是负责调用基类的默认构造函数。

对一个未提供析构函数的类，如果它有基类，编译程序有时也会隐式地为之提供一个析构函数，其作用是负责调用基类的析构函数。

如果一个类D既有基类B、又有成员对象类M，则

在创建D类对象时，构造函数的执行次序为：B->M->D

当D类的对象消亡时，析构函数的执行次序为：D->M->B

2. 派生类拷贝构造函数

- 派生类的隐式拷贝构造函数（由编译程序提供）：
 - 对派生类中新定义的成员进行拷贝初始化外。
 - 调用基类的拷贝构造函数实现对基类成员的初始化。
- 派生类自定义的拷贝构造函数：
 - 在默认情况下调用基类的默认构造函数对基类成员初始化。
 - 需要在“成员初始化表”中显式地指出调用基类的拷贝构造函数来实现对基类成员的初始化。

```
1 class A
2 {   int x;
3     .....
4 };
5 class B: public A
6 {   int y;
7 public:
8     B() { ..... }
9     B(const B& b) //调用A类的默认构造函数
10    {   y = b.y; //对派生类新定义的成员进行初始化
11    }
12    .....
13 };
14 B b1;
15 B b2(b1);
16
```

3. 派生类对象的赋值操作

- 派生类隐式的赋值操作：
 - 对派生类成员进行赋值
 - 调用基类的赋值操作对基类成员进行赋值
- 派生类自定义的赋值操作：
 - 不会自动调用基类的赋值操作
 - 需要在自定义的赋值操作符重载函数中显式地指出调用基类的赋值操作

```
1 class A
2 {   int x;
3     .....
4 };
5
6 class B: public A
7 {   int y;
8 public:
9     B& operator =(const B& b)
10    {   if (&b == this) return *this; //防止自身赋值。
11        *(A*)this = b; //调用基类的赋值操作符对基类成员
12        //进行赋值。也可写成: this->A::operator =(b);
13        y = b.y; //对派生类新定义的成员赋值
14        return *this;
15    }
```



```
16      .....
17  };
18  .....
19  B b1,b2;
20  b1 = b2;
21
```